

CS6.302 Software System Development – Monsoon 2026

JavaScript Practice Questions

NOTE: All questions are self-explanatory. Make appropriate assumptions and are open to your interpretation

1. **Heading Text Replacement:** Write an HTML structure containing an `<h1>` element with the text "Welcome!" and an ID of "title", along with a button. Write a JavaScript function called `changeText()` that modifies the heading's `textContent` to "Hello, World!" when the button is clicked.
2. **Inline Style Manipulation:** Create a `<div>` with the ID "box" that has an initial width of 100px, height of 100px, and a background-color of blue. Write a function called `changeColor()` that updates the box's background color to red via JavaScript's `.style` property upon clicking a button.
3. **Multi-line Alert Notification:** Write a JavaScript line using `alert()` that prints a greeting across two lines using an escape sequence (e.g., "Hello!" on the first line and "Welcome to our website." on the second line).
4. **Dynamic Alert Concatenation:** Declare a variable `userName = "John"`. Write an `alert()` statement that concatenates this variable to display: "Hello, John! Welcome back."
5. **Button-Triggered Alert:** Write a JavaScript function `showAlert()` that displays the message "Button clicked!" in an alert box, and bind it to a `<button>` using the `onclick` attribute.
6. **Form Validation Alert:** Write an `if` condition in JavaScript that checks whether an `<input>` field with the ID "name" is empty (`""`). If it is empty, trigger an alert stating "Name field cannot be empty!".
7. **Prompt Box with Default Value:** Write a script using `prompt()` that asks the user "Please enter your name" and supplies the default placeholder value "John Doe". Store the response in a variable named `name`. If the user does not cancel the prompt, log "Hello, [name]" to the console; otherwise, log "User cancelled the prompt."
8. **Basic Confirmation Check:** Write a script that uses `confirm()` with the message "Do you want to proceed?". Log "User clicked OK." to the console if the user confirms, and "User clicked Cancel." if they decline.
9. **Inline Form Submission Guard:** Write an HTML `<form>` tag that includes an `onsubmit` inline attribute using `confirm()`. The form should ask "Are you sure you want to submit this form?" and cancel the submission if the user clicks "Cancel".

10. **Action Verification Function:** Write a function `deleteItem()` that prompts the user with `confirm("Are you sure you want to delete this item?")`. If confirmed, log "Item deleted." to the console; otherwise, log "Action canceled."
11. How can you write an `if...else if...else` script that checks whether an integer `num` is positive, negative, or zero?
12. How would you write a script to evaluate a numeric score marks (0–100) and output letter grades ("A" for ≥ 90 , "B" for 80–89, "C" for 70–79, and "F" below 70)?
13. How can you construct a script using `if...else` statements to determine whether a given year `year` is a leap year based on the standard divisibility rules?
14. How would you implement a `switch` statement that checks a `lightColor` variable ("red", "yellow", "green") and logs the appropriate driving action, while handling unexpected values with a default case?
15. How can you use a `switch` statement with grouped cases (omitting `break` between matching labels) to output the season for a given month number (1–12)?
16. How do you write a standard `for` loop to print the multiplication table of 7 from 7×1 to 7×10 ?
17. How can you iterate over an array of numbers like `[12, 45, 19, 80, 5]` using a `for` loop to calculate and display the total sum?
18. How would you write a reverse `for` loop that iterates backward through a string such as "JavaScript" to print each character in reverse order?
19. How can you implement a `while` loop that counts down from 10 to 1 and logs "Liftoff!" upon completion?
20. How do you construct a `do...while` loop that simulates rolling a 6-sided die, logs each roll, and continues rolling until a 6 appears?
21. **Variable Scope & Hoisting:** Write a function demonstrating the difference between `var` and `let` inside an `if` block, showing how variable shadowing and block scope behave when accessed outside the block.
22. **Preventing Mutation:** Create an object representing a user profile (name, age, settings). Use `Object.freeze()` to make it immutable, and write a test demonstrating that modifying existing properties or adding new ones fails silently or throws an error in strict mode.
23. **Array Transformations:** Given an array of numeric prices `[12.5, 40.0, 99.99, 5.0]`, write a script using `map()` to return a new array where every price includes a 10% tax formatted to two decimal places.
24. **Array In-Place Modification vs. Shallow Copying:** Given an array `fruits = ["Apple", "Banana", "Cherry"]`, write a snippet that uses `push()` and

`shift()` to mutate the original array, followed by a separate snippet that uses `slice()` to extract the middle element without modifying the original array.

25. **Array Deduplication via Set:** Write a function `removeDuplicates(arr)` that accepts an array with repeated primitive values (e.g., `[1, 2, 2, 3, 4, 4, 5]`) and returns a new array with all duplicates removed using a Set.
26. **Set Membership Check:** Create a Set of allowed roles (`"admin"`, `"editor"`, `"viewer"`). Write a function `isAuthorized(role)` that takes a string input and efficiently checks whether that role is authorized using `.has()`.
27. **Map with Non-String Keys:** Create a Map that uses two distinct user object references as keys and stores their last login timestamps as the values. Retrieve and log the timestamp for one of the specific object references.
28. **Word Frequency Counter Using Map:** Given a sentence string, write a script that splits the string into words and populates a Map where each key is a word and the value is the count of how many times that word appeared.
29. **Nested Array Traversal:** Given the multidimensional array `const matrix = [ ["a", "b"], ["c", "d"], ["e", "f"] ]`, write code to extract the element `"d"`, and then iterate through the nested structure using nested loops to print each element.
30. **Map Iteration:** Create a Map containing four key-value pairs representing country codes and country names. Use a `for...of` loop with array destructuring (`[key, value]`) to print each entry in the format `"Code: [key], Country: [value]"`.
31. **Function Declarations vs. Expressions:** What is the fundamental difference in behavior between defining a function using a declaration (`function add() {}`) versus a function expression (`const add = function() {}`), particularly regarding hoisting?
32. **Arrow Function Limitations:** Why can arrow functions not be used as constructors with the `new` keyword, and how does their handling of `this` differ from regular functions?
33. **Default Return Behavior:** What value does a JavaScript function return by default if the return statement is omitted or contains no expression?
34. **Explicit Binding:** Explain the differences between the `call()`, `apply()`, and `bind()` methods when setting the execution context (`this`) of a function.
35. **Contextual this in Strict Mode:** How does the value of `this` inside a standalone regular function call differ between sloppy (non-strict) mode and strict mode (`'use strict'`)?
36. **Simulating Private Variables:** How can private variables be encapsulated inside a constructor function prior to the introduction of ES2022 private fields (`#`), and what is a privileged method in this context?

37. **ES2022 Private Class Fields:** How do private fields declared with the # prefix in ES6+ classes prevent access from outside the class instance compared to properties assigned via this?
38. **Static Members:** What is the purpose of static variables and methods in JavaScript classes, and why can they not be accessed directly through an instance of that class?
39. **finally Execution Guarantee:** If a try or catch block executes a return statement, will the finally block still execute? Explain the execution flow.
40. **Error Object Handling:** What parameters does the catch block receive when an error is thrown, and what built-in properties are commonly inspected on that error object?

41. Consider the following snippet:

```
function regularFn() {  
  return 42;  
}  
  
async function asyncFn() {  
  return 42;  
}  
  
console.log(regularFn());  
console.log(asyncFn());
```

What will each `console.log()` statement output to the terminal, and why do they differ?

42. **Promise Lifecycle & States:** Write a script that creates a custom Promise using the Promise constructor. It should simulate an asynchronous task using `setTimeout`:
- If a boolean flag `isSuccess` is true, fulfill the promise after 1 second with the message "Data loaded".
 - If `isSuccess` is false, reject the promise after 1 second with an `Error("Network failure")`. Consume this promise using `.then()`, `.catch()`, and `.finally()`.
43. **Refactoring .then() to async/await:** Rewrite the following Promise chain into an async function using `await` and a standard `try...catch` block for error handling:

```
function loadPost() {
  fetch("https://jsonplaceholder.typicode.com/posts/1")
    .then((res) => res.json())
    .then((data) => console.log(data))
    .catch((err) => console.error("Fetch failed:", err));
}
```

44. **Sequential vs. Parallel Execution:** Suppose you have two asynchronous functions, `fetchUserProfile()` and `fetchUserPosts()`, each returning a Promise:

- a. Write a script to execute them **sequentially** using `await`.
- b. Write a second script to execute them **in parallel** so both requests run concurrently, resolving only when both have settled using `Promise.all()`.

45. **Syntax & await Scope Constraints:** Will the following code run successfully in standard JavaScript? If not, identify the syntax or runtime error and explain how to fix it:

```
function processUser() {
  const response = await
  fetch("https://jsonplaceholder.typicode.com/users/1");
  const user = await response.json();
  return user.name;
}
```

46. How do you safely parse an untrusted JSON string into a JavaScript object without crashing the application if the data is malformed?

47. Why must an object or array be converted using `JSON.stringify()` before storing it in `localStorage`, and what happens if you store it directly?

48. What is the standard JavaScript technique used to delete an existing cookie using `document.cookie`?

49. Write a small JavaScript script that saves and manages user settings using both `document.cookie` and `localStorage`.

a. **Cookie Task:** Set a cookie named `session_id` with the value `abc123xyz` that expires in **7 days** and has a path of `/`.

b. **LocalStorage Task:**

- a. Store a user preferences object: `{ theme: "dark", fontSize: 16 }`.
- b. Retrieve the stored preferences, parse them, and print the `fontSize` property to the console.

c. **Cleanup:** Remove the `session_id` cookie by expiring it, and remove the preferences item from `localStorage`.

50. Add a JS feature of your choice to your https://serc-iiith.github.io/CS6302_M26.html course website.